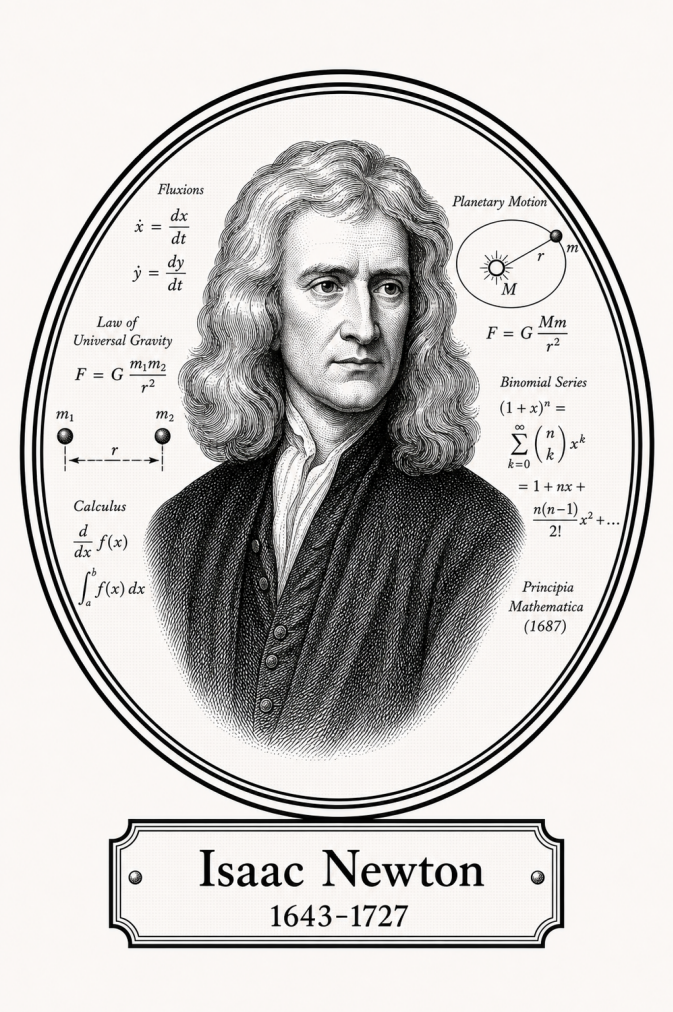
From Turing to Carmack

Applied and Computational Mathematics

Numerical Foundations



Isaac Newton

1643-1727

Self-Study Notes

William Sollers

July 1, 2026

For every reader learning to make mathematics their own.

Contents

Chapter 1

Numerical Analysis

1.1 IEEE 754 Floating-Point Formats

Toolkit — Floating-Point Representation

This section develops the structural language of IEEE 754 binary floating-point representation. The key ideas are:

- every binary floating-point datum is organized into sign, exponent, and fraction fields,
- the exponent field determines whether the encoding is normalized, subnormal, zero, infinity, or NaN,
- the fraction field determines the significant bits of the represented value,
- each standard format is determined by its total bit width together with its exponent width, fraction width, and exponent bias.

IEEE 754 Floating-Point Formats

Definition - IEEE 754 Binary Floating-Point Datum

Definition 1.1.1 (IEEE 754 Binary Floating-Point Datum)**.** An *IEEE 754 binary floating-point datum* is a binary encoding partitioned into three fields:

- a *sign field* consisting of one bit,
- an *exponent field* consisting of a fixed number of bits,
- a *fraction field* consisting of a fixed number of bits.

Its interpretation depends on the exponent-field pattern and the fraction-field pattern.

Remark (Standard quantified statement)*.*

$$\text{IEEEBinaryFloatDatum}(x) \iff \text{HasSignField}(x) \wedge \text{HasExponentField}(x) \wedge \text{HasFractionField}(x).$$

Remark (Interpretation)*.* IEEE 754 does not interpret a floating-point bit string as a plain binary integer. Instead, the sign field determines sign, the exponent field determines scale and special cases, and the fraction field records the significant bits of the encoded value.

Definition - Normalized IEEE 754 Binary Number

Definition 1.1.2 (Normalized IEEE 754 Binary Number)**.** Let an IEEE 754 binary floating-point format have exponent bias bias. A finite encoded datum is called *normalized* if its exponent field is neither all zeros nor all ones.
If s is the sign bit, if e is the unsigned integer determined by the exponent field, and if f is the binary fraction determined by the fraction field, then the represented value is

$$(-1)^s (1.f)_2 2^{e-\text{bias}}.$$

Remark (Standard quantified statement)*.*

$$\text{NormalizedIEEEBinaryNumber}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{ExponentFieldNeitherAllZerosNorA}$$

Remark (Interpretation)*.* In a normalized encoding, the leading binary digit of the significand is implicitly 1. This hidden leading bit is what gives normalized numbers one more bit of effective precision than the stored fraction field alone.

Definition - Subnormal IEEE 754 Binary Number

Definition 1.1.3 (Subnormal IEEE 754 Binary Number)**.** Let an IEEE 754 binary floating-point format have exponent bias bias. A finite encoded datum is called *subnormal* if its exponent field is all zeros and its fraction field is not all zeros.
If s is the sign bit and if f is the binary fraction determined by the fraction field, then the represented value is

$$(-1)^s (0.f)_2 2^{1-\text{bias}}.$$

Remark (Standard quantified statement).

$$\text{SubnormalIEEEBinaryNumber}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{ExponentFieldAllZeros}(x) \wedge \neg \text{Fractio}$$

Remark (Interpretation). Subnormal numbers fill the gap between zero and the smallest positive normalized number. They preserve gradual underflow by allowing the leading significand bit to be 0 instead of the implicit 1 used for normalized numbers.

Definition - IEEE 754 Infinity Encoding

Definition 1.1.4 (IEEE 754 Infinity Encoding)**.** An IEEE 754 binary floating-point datum encodes *infinity* if its exponent field is all ones and its fraction field is all zeros. The sign bit determines whether the represented value is $+\infty$ or $-\infty$.

Remark (Standard quantified statement).

$$\text{IEEEInfinityEncoding}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{ExponentFieldAllOnes}(x) \wedge \text{FractionFieldAllZe}$$

Remark (Interpretation). Infinity is a distinguished non-finite encoding used to represent overflow and certain limit-like arithmetic results. The sign field still matters, so IEEE 754 distinguishes positive infinity from negative infinity.

Definition - IEEE 754 NaN Encoding

Remark (Dependencies).

- IEEE 754 Binary Floating-Point Datum

Definition 1.1.5 (IEEE 754 NaN Encoding)**.** An IEEE 754 binary floating-point datum encodes *NaN* (Not a Number) if its exponent field is all ones and its fraction field is not all zeros.

Remark (Standard quantified statement).

$$\text{IEEENaNEncoding}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{ExponentFieldAllOnes}(x) \wedge \neg \text{FractionFieldAllZe}$$

Remark (Interpretation). A NaN encoding represents an undefined or invalid numerical result rather than a real number. For example, NaNs arise from operations such as invalid indeterminate forms or from the propagation of an already invalid datum through later computations.

Definition - IEEE 754 Binary16 Format

Definition 1.1.6 (IEEE 754 Binary16 Format)**.** The *IEEE 754 binary16 format* is the 16-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 5 exponent bits,
- 10 fraction bits,
- exponent bias 15.

For a normalized finite binary16 datum, the represented value is

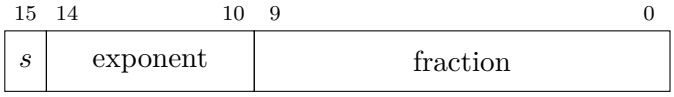
$$(-1)^s(1.f)_2 2^{e-15}.$$

Remark (Standard quantified statement)*.*

$$\text{Binary16}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{BitWidth}(x,16) \wedge \text{SignWidth}(x,1) \wedge \text{ExponentWidth}(x,5) \wedge \text{I}$$

Remark (Interpretation)*.* Binary16 is commonly called *half precision*. It is useful when reduced storage and bandwidth matter more than wide dynamic range or high precision.

Remark (Bit layout)*.*



Definition - IEEE 754 Binary32 Format

Definition 1.1.7 (IEEE 754 Binary32 Format)**.** The *IEEE 754 binary32 format* is the 32-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 8 exponent bits,
- 23 fraction bits,
- exponent bias 127.

For a normalized finite binary32 datum, the represented value is

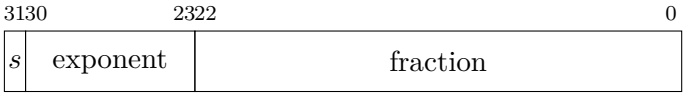
$$(-1)^s(1.f)_2 2^{e-127}.$$

Remark (Standard quantified statement)*.*

$$\text{Binary32}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{BitWidth}(x,32) \wedge \text{SignWidth}(x,1) \wedge \text{ExponentWidth}(x,8) \wedge \text{I}$$

Remark (Interpretation)*.* Binary32 is commonly called *single precision*. It is the standard 32-bit floating-point format used broadly in scientific computing, computer graphics, simulation, and machine learning.

Remark (Bit layout).



Definition - IEEE 754 Binary64 Format

Definition 1.1.8 (IEEE 754 Binary64 Format)**.** The *IEEE 754 binary64 format* is the 64-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 11 exponent bits,
- 52 fraction bits,
- exponent bias 1023.

For a normalized finite binary64 datum, the represented value is

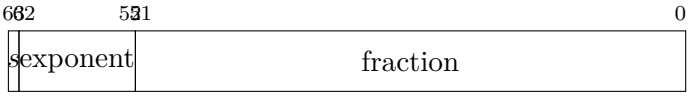
$$(-1)^s(1.f)_2 2^{e-1023}.$$

Remark (Standard quantified statement).

$$\text{Binary64}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{BitWidth}(x, 64) \wedge \text{SignWidth}(x, 1) \wedge \text{ExponentWidth}(x, 11) \wedge$$

Remark (Interpretation). Binary64 is commonly called *double precision*. It is the standard high-precision floating-point format for much of scientific and engineering computation.

Remark (Bit layout).



Definition - IEEE 754 Binary128 Format

Definition 1.1.9 (IEEE 754 Binary128 Format)**.** The *IEEE 754 binary128 format* is the 128-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 15 exponent bits,
- 112 fraction bits,
- exponent bias 16383.

For a normalized finite binary128 datum, the represented value is

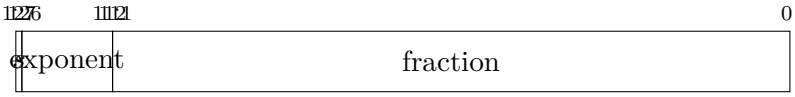
$$(-1)^s(1.f)_2 2^{e-16383}.$$

Remark (Standard quantified statement).

$$\text{Binary128}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{BitWidth}(x, 128) \wedge \text{SignWidth}(x, 1) \wedge \text{ExponentWidth}(x, 15$$

Remark (Interpretation). Binary128 is commonly called *quadruple precision*. It provides substantially greater range and precision than binary64 at substantially greater storage and computational cost.

Remark (Bit layout).



Remark (Summary). The four standard IEEE 754 basic binary interchange formats considered here are:

binary16 $(1, 5, 10)$, binary32 $(1, 8, 23)$, binary64 $(1, 11, 52)$, binary128 $(1, 15, 112)$,

where each triple records

$$(\text{sign bits},\ \text{exponent bits},\ \text{fraction bits}).$$

1.2 Interval Arithmetic

Box (Interval) Arithmetic in $\mathbb{R}^n$ - Quick Reference

Operation	Rule
Box (product interval)	$[a,b] := \{x \in \mathbb{R}^n : a \le x \le b\}$
Containment	$[a,b] \subseteq [c,d] \iff c \le a \ \wedge\ b \le d$
Minkowski sum	$[a,b] \oplus [c,d] = [a+c,\ b+d]$
Difference	$[a,b] \ominus [c,d] = [a-d,\ b-c]$
Scalar mult. $(\lambda \ge 0)$	$\lambda[a,b] = [\lambda a,\ \lambda b]$
Scalar mult. $(\lambda < 0)$	$\lambda[a,b] = [\lambda b,\ \lambda a]$
Coordinatewise product	$[a,b] \odot [c,d] = \prod_i [\min S_i, \max S_i]$
Norm bound (Euclidean)	$\|x\|_2 \le \max\{\|a\|_2, \|b\|_2\}$ for $x \in [a,b]$ if $0 \le a \le b$

Interval Arithmetic - Quick Reference

Operation	Rule
Addition	$[a,b] + [c,d] = [a+c,\ b+d]$
Subtraction	$[a,b] - [c,d] = [a-d,\ b-c]$
Multiplication	$[a,b] \cdot [c,d]$ (case analysis)
Containment	$[a,b] \subseteq [c,d] \iff c \le a, b \le d$

1.2.1 Interval Arithmetic

Definition (Interval Addition)

Definition 1.2.1 (Interval Addition)**.** For closed intervals $[a,b]$ and $[c,d]$, their *sum* is

$$[a,b] + [c,d] \ = \ [a+c,\ b+d].$$

Remark (Dependencies).

- Closed Interval
- Addition on $\mathbb{R}$

Definition (Interval Subtraction)

Definition 1.2.2 (Interval Subtraction)**.** For closed intervals $[a, b]$ and $[c, d]$, their *difference* is

$$[a, b] - [c, d] \;=\; [a - d,\; b - c].$$

Remark (Dependencies).

- Closed Interval
- Subtraction on $\mathbb{R}$

Definition (Interval Multiplication)

Definition 1.2.3 (Interval Multiplication)**.** For closed intervals $[a, b]$ and $[c, d]$, their *product* is

$$[a, b] \cdot [c, d] \;=\; \big[\min(ac, ad, bc, bd),\; \max(ac, ad, bc, bd)\big].$$

Remark (Dependencies).

- Closed Interval
- Multiplication on $\mathbb{R}$
- Order on $\mathbb{R}$

Definition (Interval Containment)

Definition 1.2.4 (Interval Containment)**.** $[a, b] \subseteq [c, d]$ if and only if $c \leq a$ and $b \leq d$.

Remark (Dependencies).

- Closed Interval
- Subset
- Order on $\mathbb{R}$

Remark (Motivation). Interval arithmetic provides a framework for rigorous numerical computation with guaranteed error bounds. A real number $x \in [a, b]$ means the computed value may lie anywhere in the interval; operations propagate these bounds.

Remark (Status). *This section is a stub. A full treatment – the sub-distributivity law, dependency problem, applications to root-finding, and connections to compactness – will be developed in a future revision.*

1.2.2 Interval Arithmetic in $\mathbb{R}^n$

Definition (Box / Product Interval)

Definition 1.2.5 (Box / Product Interval)**.** Let $a, b \in \mathbb{R}^n$ with $a \le b$ coordinatewise (i.e., $a_i \le b_i$ for all i). The *box* (or *interval vector*) determined by (a, b) is

$$[a, b] \;:=\; \{x \in \mathbb{R}^n : a \le x \le b\} \;=\; \prod_{i=1}^{n} [a_i, b_i].$$

Remark (Dependencies).

- Cartesian Product
- Closed Interval
- Order on $\mathbb{R}$

Definition (Box Containment)

Definition 1.2.6 (Box Containment)**.** For $a \le b$ and $c \le d$ in $\mathbb{R}^n$, we write $[a, b] \subseteq [c, d]$ iff

$$(\forall x \in \mathbb{R}^n)\ \big(x \in [a, b] \Rightarrow x \in [c, d]\big).$$

Equivalently (and most useful computationally),

$$[a, b] \subseteq [c, d] \quad \Longleftrightarrow \quad c \le a \ \wedge\ b \le d \qquad \text{(coordinatewise)}.$$

Remark (Dependencies).

- Box / Product Interval
- Subset
- Order on $\mathbb{R}$

Definition (Minkowski Addition of Boxes)

Definition 1.2.7 (Minkowski Addition of Boxes)**.** For boxes $X = [a, b]$ and $Y = [c, d]$ in $\mathbb{R}^n$, their *Minkowski sum* is

$$X \oplus Y \;:=\; \{x + y : x \in X,\ y \in Y\}.$$

For boxes, this closes in the class of boxes and satisfies the endpoint rule

$$[a, b] \oplus [c, d] \;=\; [a + c,\ b + d].$$

Remark (Dependencies).

- Box / Product Interval

- Addition on $\mathbb{R}$

Definition (Box Subtraction / Difference)

Definition 1.2.8 (Box Subtraction / Difference)**.** For boxes $X = [a, b]$ and $Y = [c, d]$ in $\mathbb{R}^n$, define

$$X \ominus Y \;:=\; \{x - y : x \in X,\; y \in Y\}.$$

Then

$$[a, b] \ominus [c, d] \;=\; [a - d,\; b - c].$$

Remark (Dependencies).

- Box / Product Interval
- Subtraction on $\mathbb{R}$

Definition (Scalar Multiplication of a Box)

Definition 1.2.9 (Scalar Multiplication of a Box)**.** For $\lambda \in \mathbb{R}$ and a box $[a, b] \subseteq \mathbb{R}^n$, define

$$\lambda[a, b] \;:=\; \{\lambda x : x \in [a, b]\}.$$

Then

$$\lambda[a, b] = \begin{cases} [\lambda a,\; \lambda b], & \lambda \geq 0, \\ [\lambda b,\; \lambda a], & \lambda < 0, \end{cases}$$

where the inequalities are coordinatewise.

Remark (Dependencies).

- Box / Product Interval
- Multiplication on $\mathbb{R}$
- Order on $\mathbb{R}$

Definition (Coordinatewise / Hadamard Product of Boxes)

Definition 1.2.10 (Coordinatewise / Hadamard Product of Boxes)**.** For $x, y \in \mathbb{R}^n$, write $(x \odot y)_i := x_i y_i$. For boxes $X = [a, b]$ and $Y = [c, d]$, define

$$X \odot Y \;:=\; \{x \odot y : x \in X,\; y \in Y\}.$$

Then $X \odot Y$ is again a box, computed coordinatewise:

$$[a, b] \odot [c, d] \;=\; \prod_{i=1}^{n} \Big[\min S_i,\; \max S_i\Big], \qquad S_i := \{a_i c_i,\; a_i d_i,\; b_i c_i,\; b_i d_i\}.$$

Remark (Dependencies).

- Box / Product Interval
- Multiplication on $\mathbb{R}$
- Order on $\mathbb{R}$

Remark (Interpretation: intervals as guaranteed enclosures). A box $X = [a, b] \subseteq \mathbb{R}^n$ represents a *set of possible vectors* (e.g., a computed vector plus uncertainty bounds). The operations above are *set operations* (Minkowski sum/difference, scalar image), so the result is a *guaranteed enclosure* of all possible true outcomes under the corresponding vector operation.

Remark (Norm bounds and why boxes are convenient). For many applications you also want quick bounds on a norm over a box. For example, if $0 \leq a \leq b$ coordinatewise, then for every $x \in [a, b]$ we have $0 \leq x \leq b$ and hence $\|x\|_2 \leq \|b\|_2$. More generally, bounding norms over a box can be reduced to checking finitely many "corners" when the objective is coordinatewise monotone.

Remark (Status). *This section is a stub. A full treatment – interval extensions of general functions $f : \mathbb{R}^n \to \mathbb{R}^m$, the dependency problem, sub-distributivity, natural vs. centered (affine) forms, and connections to compactness and continuity – will be developed in a future revision.*

1.3 Numerical Geometry

1.3.1 Week 1: Points, Vectors, and Floating-Point Geometry

Reading checklist.

- Atkinson, Chapter 1
- Solomon, Chapter 1 selectively
- Solomon, Chapter 2 selectively
- Vince, Chapter 7
- Existing Volume VII IEEE 754 notes
- Existing Volume VII interval arithmetic notes

Remark (Project placeholder). `lra-numerical-analysis/projects/point-vector-01`

Remark (Handwritten notes placeholder). TODO: Add handwritten-note extraction here after study.

1.3.2 Week 2: Linear Interpolation and Affine Sampling

Reading checklist.

- Salomon, Chapter 1 sections 1.1–1.3
- Salomon, Chapter 2 section 2.1
- Vince, Chapter 13
- Piegl–Tiller, Chapter 1 skim only
- Piegl–Tiller, Chapter 2 preview only

Remark (Project placeholder). `lra-numerical-analysis/projects/linear-interpolation-02`

Remark (Handwritten notes placeholder). TODO: Add handwritten-note extraction here after study.

1.3.3 Projects

- `point-vector-01`: planned
- `linear-interpolation-02`: planned
- `bezier-curve-03`: deferred
- `bspline-basis-04`: deferred
- `nurbs-curve-05`: deferred

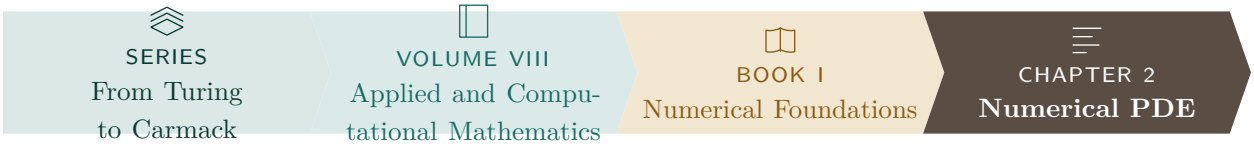
1.3.4 Exercises

- Week 1 exercises: planned
- Week 2 exercises: planned
- TODO: Add exercises after handwritten notes are extracted.

Proofs

Chapter 2

Numerical PDE



2.1 Numerical PDE

Remark (Planned content). Active mathematical content has not yet been added.

Proofs

Bibliography